

B.N.M. Institute of Technology

An Autonomous Institution under VTU
Department of Information Science and Engineering
Big Data Lab Internals

1. Develop a MapReduce program to calculate the frequency of a given word in each file.

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context
            ) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text,IntWritable,Text,IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
            Context context
            ) throws IOException, InterruptedException {

            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Steps for BDA Map Reduce program -1 Execution

Steps for BDA Map Reduce program -1 Execution

- 1) Open Eclipse
- 2) File->New->Java project
- 3) Write click on the project->New->class (Name it as Mapper class)
- 4) Write click on the project->New->class (Name it as Reducer class)
- 5) Write click on the project->New->class (Name it as Driver class- Example: MaxTemp class)
- 6) Create a Text file for Input with related input data.
- 7) Go to File->Export->Java->Jar File-> Give name for the jar file and click on finish(Example: MaxTemp.jar)
- 8) Open Terminal and execute the following commands
- i) copying the file from local file system to HDFS
hadoop fs -put Name_of_the_input_file New_filename
Example: hadoop fs -put temp.txt Maxtemp.txt (temp.txt- File in local file system
Maxtemp.txt- File in HDFS)
- ii) Execute the Mapreduce job using jar
hadoop jar name_of_the_jar_file_exported class_name Input_file_name_in_HDFS Directory_name(to store output)
Example: hadoop jar MaxTemp.jar MaxTemp Maxtemp.txt dir1 (Execution)
- iii) Display the output stored in directory
hadoop fs -cat directory_name/part-r-00000
Example: hadoop fs -cat dir1/part-r-00000/*

2. Develop a MapReduce program to find the maximum temperature in each year.

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaxTemp {

    public static class MaxTempMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        private Text year = new Text();
        private IntWritable temperature = new IntWritable();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            String[] data = line.split(",");

            String yr = data[0];
            int temp = Integer.parseInt(data[1]);

            year.set(yr);
            temperature.set(temp);

            context.write(year, temperature);
        }
    }

    public static class MaxTempReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {

        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
            Context context)
            throws IOException, InterruptedException {

            int maxTemperature = Integer.MIN_VALUE;

            for (IntWritable val : values) {
                maxTemperature = Math.max(maxTemperature, val.get());
            }

            result.set(maxTemperature);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Maximum Temperature");

        job.setJarByClass(MaxTemp.class);

        job.setMapperClass(MaxTempMapper.class);
        job.setReducerClass(MaxTempReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

3. Develop a MapReduce to analyze weather data set and print whether the day is shinny or cool day.

```
import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;

public class WeatherAnalysis {

    public static class MaxTemperatureMapper extends
        Mapper<LongWritable, Text, Text, Text> {

        public static final int MISSING = 9999;

        @Override
        public void map(LongWritable arg0, Text Value, Context context)
            throws IOException, InterruptedException {

            String line = Value.toString();

            if (!(line.length() == 0)) {

                String date = line.substring(6, 14);

                float temp_Max = Float.parseFloat(line.substring(39, 45).trim());

                float temp_Min = Float.parseFloat(line.substring(47, 53).trim());

                if (temp_Max > 30.0) {

                    context.write(new Text("The Day is Hot Day :" + date),
                        new Text(String.valueOf(temp_Max)));
                }

                if (temp_Min < 15) {

                    context.write(new Text("The Day is Cold Day :" + date),
                        new Text(String.valueOf(temp_Min)));
                }
            }
        }
    }

    public static class MaxTemperatureReducer extends
        Reducer<Text, Text, Text, Text> {

        public void reduce(Text Key, Iterator<Text> Values, Context context)
            throws IOException, InterruptedException {

            String temperature = Values.next().toString();
            context.write(Key, new Text(temperature));
        }
    }

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();

        Job job = new Job(conf, "weather example");

        job.setJarByClass(WeatherAnalysis.class);

        job.setMapOutputKeyClass(Text.class);

        job.setMapOutputValueClass(Text.class);

        job.setMapperClass(MaxTemperatureMapper.class);

        job.setReducerClass(MaxTemperatureReducer.class);

        job.setInputFormatClass(TextInputFormat.class);

        job.setOutputFormatClass(TextOutputFormat.class);
```

```
    Path outputPath = new Path(args[1]);  
    FileInputFormat.addInputPath(job, new Path(args[0]));  
    FileOutputFormat.setOutputPath(job, new Path(args[1]));  
    outputPath.getFileSystem(conf).delete(outputPath);  
    System.exit(job.waitForCompletion(true) ? 0 : 1);  
}
```

4. Develop a MapReduce program to find the grades of student's.

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class StudentGrades {

    public static class GradeMapper extends Mapper<Object, Text, Text, Text> {

        private Text studentId = new Text();
        private Text result = new Text();

        public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
            String line = value.toString();
            StringTokenizer tokenizer = new StringTokenizer(line, ",");
            String id = tokenizer.nextToken();
            double sum = 0;
            int count = 0;

            while (tokenizer.hasMoreTokens()) {
                sum += Double.parseDouble(tokenizer.nextToken());
                count++;
            }

            double percentage = sum / count;

            studentId.set(id);
            result.set(Double.toString(percentage));
            context.write(studentId, result);
        }
    }

    public static class GradeReducer extends Reducer<Text, Text, Text, Text> {

        public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
            double percentage = 0;
            int count = 0;

            for (Text value : values) {
                percentage = Double.parseDouble(value.toString());
                count++;
            }

            double avgPercentage = percentage / count;
            String grade = getGrade(avgPercentage);

            context.write(key, new Text(grade));
        }

        private String getGrade(double percentage) {
            if (percentage >= 90) {
                return "A";
            } else if (percentage >= 80) {
                return "B";
            } else if (percentage >= 70) {
                return "C";
            } else if (percentage >= 60) {
                return "D";
            } else {
                return "F";
            }
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "student grades");
        job.setJarByClass(StudentGrades.class);
        job.setMapperClass(GradeMapper.class);
        job.setReducerClass(GradeReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```


5. Write queries to sort and aggregate the data in a table using HiveQL

1. Create database

```
Create database ISE;
```

2. Create table

```
Create table student (ID int, USN int, Name string, DOB int, Course string, Sem int, Percentage int) ROW FORMAT DELIMITED
```

3. Insert data into table

```
Insert into table student values(1,11,abc1,1may2000,ise,6,96);
```

//create input text file consisting of student details as above and add the location in path of input file.

4. Load Data into table

```
Load data local inpath '/path of input file/' into table student;
```

5. Display the table contents

```
Select * from student;
```

6. Use order by

```
Select * from student order by Sem asc;
```

7. Use group by

```
Select sem from student Group by Sem;
```

8. Use Distribute by

```
Select * from student distribute by Sem;
```


6. Demonstrate HIVE UDF to convert table data into upper case letter.

```
Upper.java

import org.apache.hadoop.hive.ql.exec.UDF;

import org.apache.hadoop.io.Text;

public class UPPER extends UDF {

    public Text evaluate(Text input) {

        if (input == null) {

            return null;

        }

        return new Text(input.toString().toUpperCase());

    }

}
```

Note:

Add external jar files from hive and Hadoop

Create the jar file by exporting jar file as Upper.jar

Steps to execute

Type Hive in terminal

Hive> add jar name_of_the jar

Example: hive>add jar /home/cloudera/Upper.jar

2) Create temporary function name_of_the_function as 'package_name.class_name';

Example: create temporary function upperletter as 'UPPER';

(UPPER is name of the java class)

3) Select upperletter(name), upperletter(course) from student;

(create student table and add the details to student table in hive)

7. Compute the average of a given numeric values using pig

```
dividends = load 'myfile.txt' as (A);

grouped   = GROUP dividends all;

avg        = foreach grouped generate AVG(dividends);

dump avg

save list of numbers in MYFILE.TXT

12
13
13
45.7
78.8
78.8
89

COPY THE FILE IN THE LOCAL SYSEYEM TO HADOOP.

Hadoop fs -put myfile.txt myfile.txt

Type pig in terminal and then type above comments in grunt.
```